Collaborative Pixel Canvas with Rebalancing and Recoverable History

Tanisha Upadhyay (2025201088)
Vidur Shah (2026909004)

Distributed Systems Course Project

Abstract

This project proposes a distributed, real-time 1000×1000 pixel canvas. Tile-based sharding and live migration allow drawing during rebalancing; ownership epochs and per-tile positions order updates and reject stale writes. Durable logs and coordinated snapshots support recovery and history replay, including after a shard crashes during migration.

1 Project Description

Multiple users will draw simultaneously on a shared 1000×1000 pixel grid. A browser interface will let users select colours by hexadecimal value, paint individual cells, attempt eligible undos, and clear the canvas. Each session receives a unique identifier so operations can be attributed to users.

The backend partitions the canvas into 25×25 pixel tiles, yielding 1,600 independently managed tiles. Separate shard processes own subsets of these tiles. The system will support adding a node and migrating tiles during active drawing without losing concurrent writes. Ownership epochs prevent a previous owner from accepting writes after transfer.

Each committed update has an operation identifier and a per-tile position. These fields establish deterministic ordering and allow duplicate deliveries to be ignored. Coordinated snapshots and durable update logs restore canvas state after process failure and reproduce its history. Restored canvas hashes and timelapse frames will be compared with a logged reference history.

2 Goals

- **Real-time collaboration:** deliver drawing updates to connected users through WebSockets.
- **Distributed sharding:** assign canvas tiles to multiple backend processes.
- **Live rebalancing:** migrate tile ownership while drawing continues and preserve concurrent writes.
- **Deterministic consistency:** order committed tile updates and reject stale owners with ownership epochs and update positions.
- **Fault recovery:** restore shard state using persistent logs, snapshots, and replay, including interrupted migrations.
- **Idempotency:** deduplicate retried and replayed operations.
- **User-aware operations:** allow up to ten undo attempts per user without overwriting a newer update by another user, and record clear operations in history.
- **Verification:** compare recovered canvas hashes and timelapse frames against the reference history.

3 System Architecture

Component	Responsibility
Browser clients	HTML, CSS, JavaScript, and Canvas API interface for drawing and viewing live updates.
FastAPI router	Manage sessions, control requests, tile routing, and ownership epochs.
Shard processes	Own assigned tiles, order and apply updates, and serve migration and recovery operations.
Persistent storage	Keep per-shard snapshots, migration progress, deduplication state, and durable update logs.

WebSockets carry live drawing updates; REST/HTTP handles control actions. Migration coordination, log replay, and hash verification operate across the components. Multiple shard processes may run locally or on the university Remote Computing Environment (RCE).

4 Consistency, Migration, and Recovery

For each tile, the current owner assigns monotonically increasing update positions within an ownership epoch. A write is accepted only by the owner for the current epoch. An operation identifier lets the shard recognize a retry and return its prior result without applying the update again.

During migration, the source records transfer progress durably, sends the tile state and relevant log position to the destination, and accounts for writes arriving during transfer. Ownership changes only after the destination is ready to continue from the agreed position. The source then rejects writes under its old epoch. Recovery reads persisted migration progress to determine whether to resume or safely complete an interrupted transfer.

Coordinated snapshots record a consistent recovery point. On restart, a shard loads its snapshot and replays later log entries in order, skipping duplicate operation identifiers. Replaying the complete logged history also produces reference canvas states and timelapse frames for verification.

5 Tools and Technologies

- **Backend:** Python and FastAPI; separate shard processes with individual ports and storage directories.
- **Communication:** WebSockets for live updates and REST over HTTP for control actions.
- **Frontend:** HTML, CSS, JavaScript, and the browser Canvas API.
- **Canvas data:** NumPy arrays of RGB `uint8` values; Pillow for snapshots and image processing.
- **Persistence:** SQLite and durable operation logs.
- **Verification:** automated Python tests, hashing, and timelapse comparison.

6 System Requirements and Scope

Development requires a standard computer with Python, the required libraries, and a modern browser. A complete canvas occupies a $1000 \times 1000 \times 3$ RGB array; each shard should store only its assigned tiles where practical. Shard processes run independently with persistent directories that survive restarts.

The frontend will be publicly hosted for multi-user access, with a suitable secure tunnel or hosted endpoint connecting it to the backend. RCE can be used to demonstrate distributed execution but is optional. Replication across independent machines is outside the initial scope.

7 Validation Plan

Tests will cover concurrent drawing, node addition, node restart, live migration, duplicate delivery, and controlled shard failure during transfer. The final demonstration will add a shard, migrate tiles while users draw, crash a shard during migration, restart it, and recover the canvas. Canvas hashes and selected timelapse frames will be compared with those generated from the logged reference history. A match provides objective evidence that migration and replay preserved the committed update sequence.

8 Expected Outcomes

The expected result is a functional collaborative canvas that demonstrates sharding, logical clocks, live migration, consistent snapshots, log replay, and idempotency. Users should observe continuous shared drawing while shard ownership changes. After a failure, restored state and replayed history should match the reference results, including recorded undo and clear operations.